Read-Write Lock

Season: 5 · Concurrency · Category: Concurrency · Level: Intermediate

Intent

- Many readers or one writer.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- Library: many readers, exclusive re-shelving.

When to Use

- Concurrent / parallel workloads.
- The design force named in Intent shows up more than once.

When Not to Use

- Pure CPU-bound work already parallelized.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.concurrency.readwritelock;

import java.util.concurrent.locks.ReentrantReadWriteLock;
import java.util.function.Supplier;

public final class CachedValue<T> {
    private final ReentrantReadWriteLock lock = new ReentrantReadWriteLock();
    private T value;

    public T get(Supplier<T> loader) {
        lock.readLock().lock();
        try {
            if (value != null) return value;
        } finally {
            lock.readLock().unlock();
        }
        lock.writeLock().lock();
        try {
            if (value == null) value = loader.get();
            return value;
        } finally {
            lock.writeLock().unlock();
        }
    }

    public void clear() {
        lock.writeLock().lock();
        try { value = null; } finally { lock.writeLock().unlock(); }
    }
```

Classic Implementation (2)

```
}
```

Modern Java 25 / Functional

- Executors.newVirtualThreadPerTaskExecutor() is the modern default for blocking tasks.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: concurrency/readwritelock/
```

Pitfalls

- Prefer java.util.concurrent and virtual threads over custom pools.
- Always interrupt/shutdown cleanly.

Interview Q&A

- Q: Virtual threads vs platform threads? A: Virtual threads excel at blocking I/O concurrency; still bound by CPU for pure compute.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Many readers or one writer.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: Library: many readers, exclusive re-shelving. [0:40] Intent — State the design force in one sentence...